Learning Objectives:

1. To understand the concept of intelligent agents in Artificial Intelligence.
2. To study different types of agents:
    a. Simple Reflex Agent
    b. Model-Based Agent
    c. Goal-Based Agent
    d. Utility-Based Agent
3. To implement **basic agent programs in Python**.
4. To observe how different agents make decisions in different situations

---

1. Introduction:

In Artificial Intelligence an agent is anything that can observe its environment and take actions to achieve a task.

An AI agent works in the following way:

Agent = Perception + Decision + Action

- **Perception** → Agent senses the environment using **sensors**
- **Decision** → Agent processes the information
- **Action** → Agent performs an action using **actuators**

Example

A robot vacuum cleaner:

- Sensor → Detects dirt on the floor
- Decision → Determines whether cleaning is needed
- Action → Cleans the floor

2. Types of AI Agents:

The main types of AI agents are:

1. Simple Reflex Agent

2. Model-Based Agent
3. Goal-Based Agent
4. Utility-Based Agent

3. Simple Reflex Agent:

A Simple Reflex Agent makes decisions **only based on the current situation**.
It does not store past information.

It follows simple **IF–THEN rules**.

Example rule:

```
IF condition THEN action
```

Example: Automatic Door Controller

If a person is detected near the door, the door opens.
If no person is detected, the door closes.

Python Implementation:

```python
def automatic_door(sensor):

    if sensor == 1:
        action = "OPEN DOOR"
    else:
        action = "CLOSE DOOR"

    return action

sensor_input = int(input("Enter Motion Sensor Status (1 = Person, 0 = No Person): "))

result = automatic_door(sensor_input)

print("Agent Action:", result)
```

4. Model-Based Agent

- A Model-Based Agent keeps **internal memory of the environment**.

- It stores information about the world and updates it after every action.
- This helps the agent work even when the environment is **not fully visible**.

Example: Vacuum Cleaner World

The vacuum cleaner must clean three rooms:

- Room A
- Room B
- Room C

The goal is that **all rooms become clean**.

Python Implementation:

```
rooms = {}

location = input("Enter Initial Location (A/B/C): ")

rooms['A'] = int(input("Enter Status of Room A (0=Clean, 1=Dirty): "))
rooms['B'] = int(input("Enter Status of Room B (0=Clean, 1=Dirty): "))
rooms['C'] = int(input("Enter Status of Room C (0=Clean, 1=Dirty): "))

for room in rooms:

    if rooms[room] == 1:
        print("Room", room, "is DIRTY -> CLEANING")
        rooms[room] = 0

    else:
        print("Room", room, "is already CLEAN")

print("Final State:", rooms)
```

Sample Output:

```
Enter Initial Location (A/B/C): A
Enter Status of Room A (0=Clean, 1=Dirty): 1
Enter Status of Room B (0=Clean, 1=Dirty): 0
Enter Status of Room C (0=Clean, 1=Dirty): 1
Room A is DIRTY -> CLEANING
Room B is already CLEAN
Room C is DIRTY -> CLEANING
Final State: {'A': 0, 'B': 0, 'C': 0}
```

5. Goal-Based Agent

- A Goal-Based Agent chooses actions that help it **reach a specific goal**.
- The agent evaluates possible actions and selects the one that leads to the goal.

Example: Robot Reaching Charging Station

A robot wants to reach the charging station.

Locations:

- RoomA
- RoomB
- ChargingStation

Goal → Reach **ChargingStation**

Python Implementation:

```python
goal = "ChargingStation"

def goal_agent(location):

    if location == goal:
        return "Robot already at Charging Station"

    elif location == "RoomA":
        return "Move from RoomA to RoomB"

    elif location == "RoomB":
        return "Move from RoomB to ChargingStation"

    else:
        return "Invalid Location"

current_location = input("Enter Robot Location (RoomA/RoomB/ChargingStation): ")

action = goal_agent(current_location)

print("Agent Action:", action)
```

Sample Output:

```
Enter Robot Location (RoomA/RoomB/ChargingStation): RoomB
Agent Action: Move from RoomB to ChargingStation
```

6. Utility-Based Agent

- A Utility-Based Agent chooses the action that gives the **highest utility (best outcome)**.
- Utility means how good a result is.

Example: Route planning system choosing the best route with least travel time.

Smart Route Planning System

Routes available:

Route	Time
Route A	25 minutes
Route B	15 minutes
Route C	20 minutes

➔ The system selects the route with minimum travel time.

Python Implementation:

```python
routes = {
    "Route A": 25,
    "Route B": 15,
    "Route C": 20
}

best_route = min(routes, key=routes.get)

print("Best Route:", best_route)
print("Travel Time:", routes[best_route], "minutes")
```

```
Best Route: Route B
Travel Time: 15 minutes
```

2nd Method:

```
routes = {
    "route1": 5,   # utility score
    "route2": 8,
    "route3": 6
}

best_route = max(routes, key=routes.get)

print("Best route is:", best_route)
```

```
Best route is: route2
```

Lab Tasks:

1. Develop a simple reflex agent program in Python for the vacuum-cleaner-world problem.

Some predefined knowledge for agent:

- Actions: Only 0 or 1 where 0 means CLEAN and 1 means DIRTY
- Rooms: Your agent have only 3 rooms (A/B/C)
- Goal state: {'A':0, 'B':0, 'C':0}
- States Sequence: A , B, C are rooms connected in following order : A→B and B→C
- Path Cost: +1 for every Action (Either from dirt to clean OR moving from one room to another)

Inputs of Agent:

1. Enter LOCATION (Initial Vacuum placement) A/B/C in capital letters.
2. Enter Status of Current Location 0/1 accordingly.
3. Vacuum Cleaner senses the status of the other rooms before performing any action, also known as Environment sensing. So, Give Status of other rooms as input. (0/1)

2. Develop a Goal-Based Agent in Python for a **maze-solving robot**.

A robot is placed in a simple maze. The robot's goal is to reach the Goal position (G) starting from Start position (S).

The maze is represented as a straight path:

S → A → B → C → G

Predefined Knowledge

- Start Position: S
- Goal Position: G
- Path: S → A → B → C → G

3. Modify the vacuum cleaner program to count how many rooms were cleaned**.**

Lab # 06 Marks distribution

	ER1	ER6	ER8
Task	3 points	3 points	4 points

Lab # 06 Rubric Evaluation Guideline:

#	Qualities & Criteria	0 < Poor <= 1	1 < Satisfactory <= 2	2 < Excellent <=3
ER1	**Task Completion**	Minimal or no program functionality was achieved.	Some tasks were completed, but the program has errors or incomplete functionalities.	All tasks were completed, and the program runs without errors.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1 < Satisfactory <= 2**	**2 < Excellent <=3**
ER6	**Program Output**	Output is inaccurate or poorly presented.	Output is mostly accurate but may lack labels, captions, or formatting.	Output is clear, accurate, and well presented with labels, captions, and proper formatting.
#	**Qualities & Criteria**	**0 < Poor <= 1**	**1.5 < Satisfactory <= 2.5**	**2.5 < Excellent <= 4**
ER8	**Question & Answer**	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.